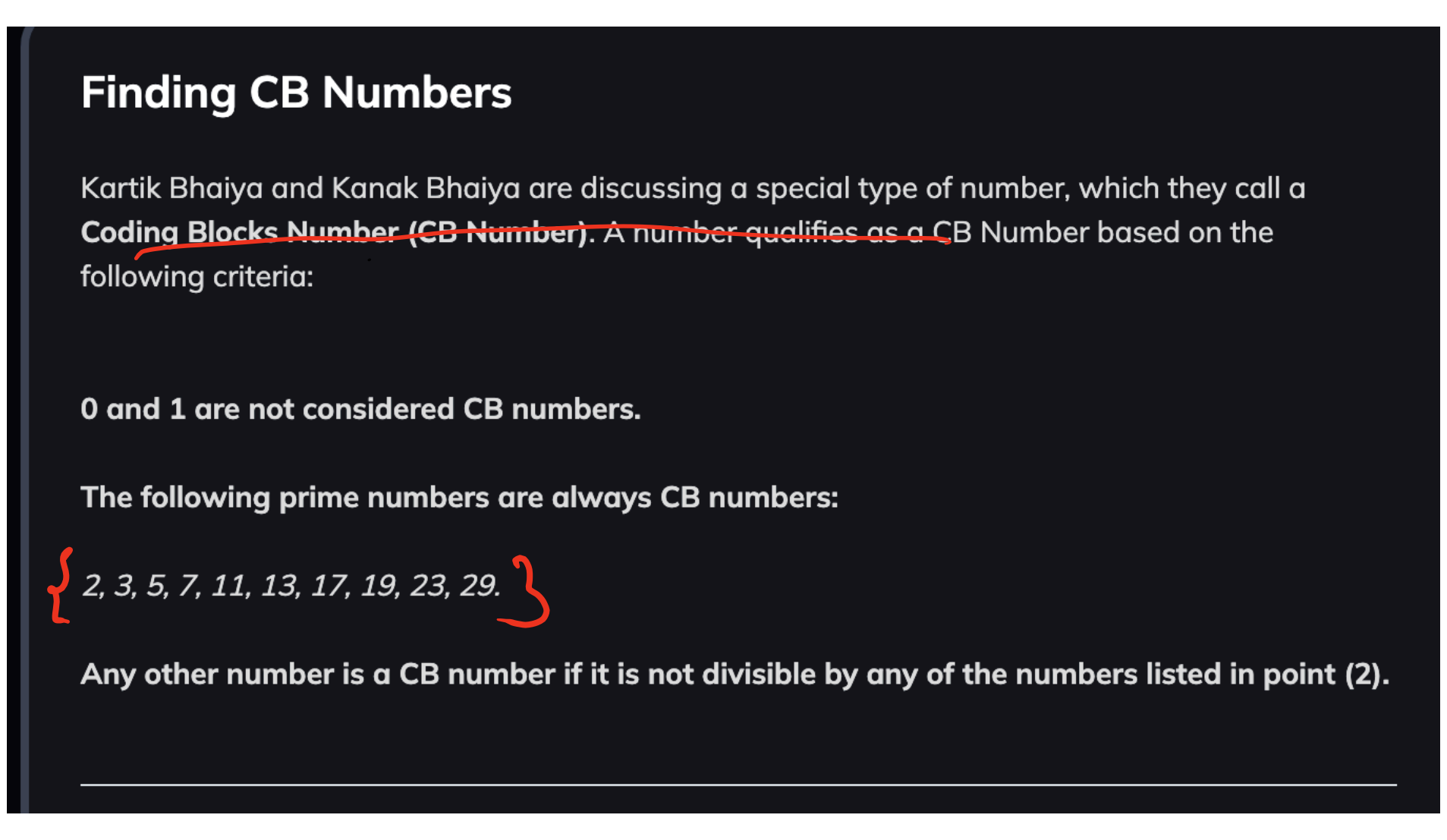
Lec-14

17/10/26 4:53 PM



Finding CB Numbers

Kartik Bhaiya and Kanak Bhaiya are discussing a special type of number, which they call a **Coding Blocks Number (CB Number)**. ~~A number qualifies as a~~ CB Number based on the following criteria:

0 and 1 are not considered CB numbers.

The following prime numbers are always CB numbers:

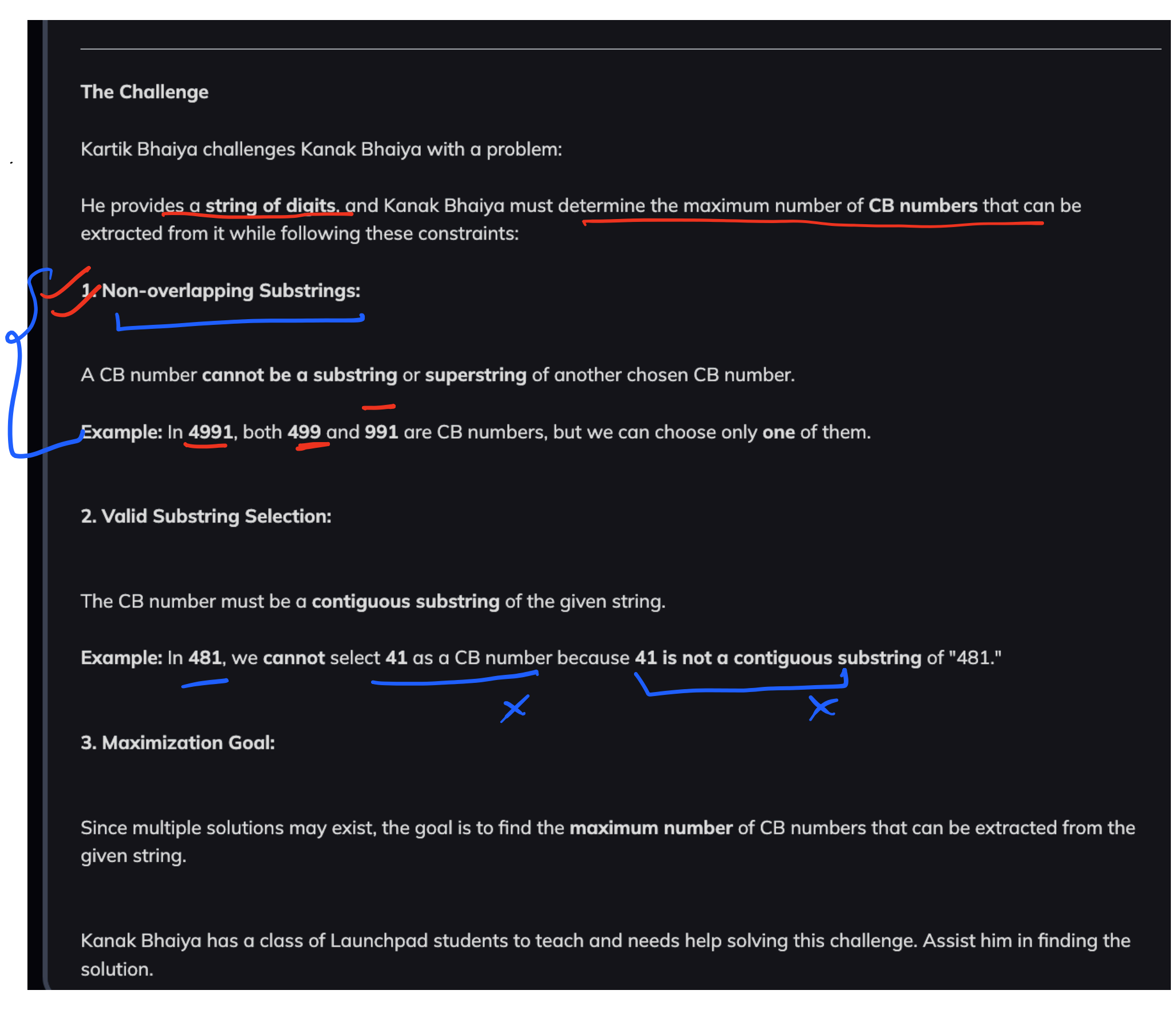
{ *2, 3, 5, 7, 11, 13, 17, 19, 23, 29.* }

Any other number is a CB number if it is not divisible by any of the numbers listed in point (2).

0,1

logic ✓

61



The Challenge

Kartik Bhaiya challenges Kanak Bhaiya with a problem:

He provides a **string of digits**, and Kanak Bhaiya must determine the maximum number of **CB numbers** that can be extracted from it while following these constraints:

1. Non-overlapping Substrings: ✓

A CB number **cannot be a substring** or **superstring** of another chosen CB number.

Example: In **4991**, both **499** and **991** are CB numbers, but we can choose only **one** of them.

2. Valid Substring Selection:

The CB number must be a **contiguous substring** of the given string.

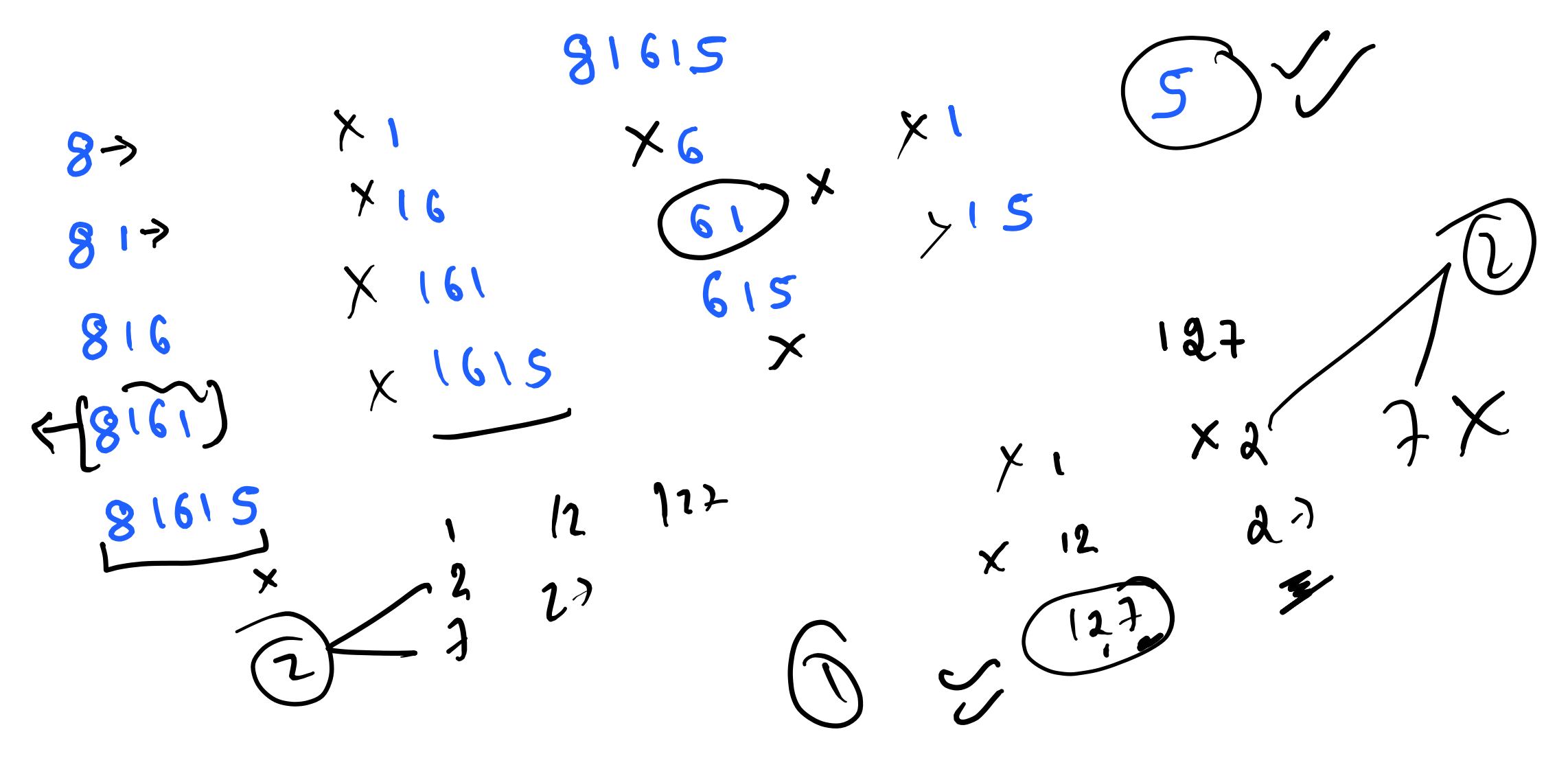
Example: In **481**, we **cannot** select **41** as a CB number because **41 is not a contiguous substring** of "481."

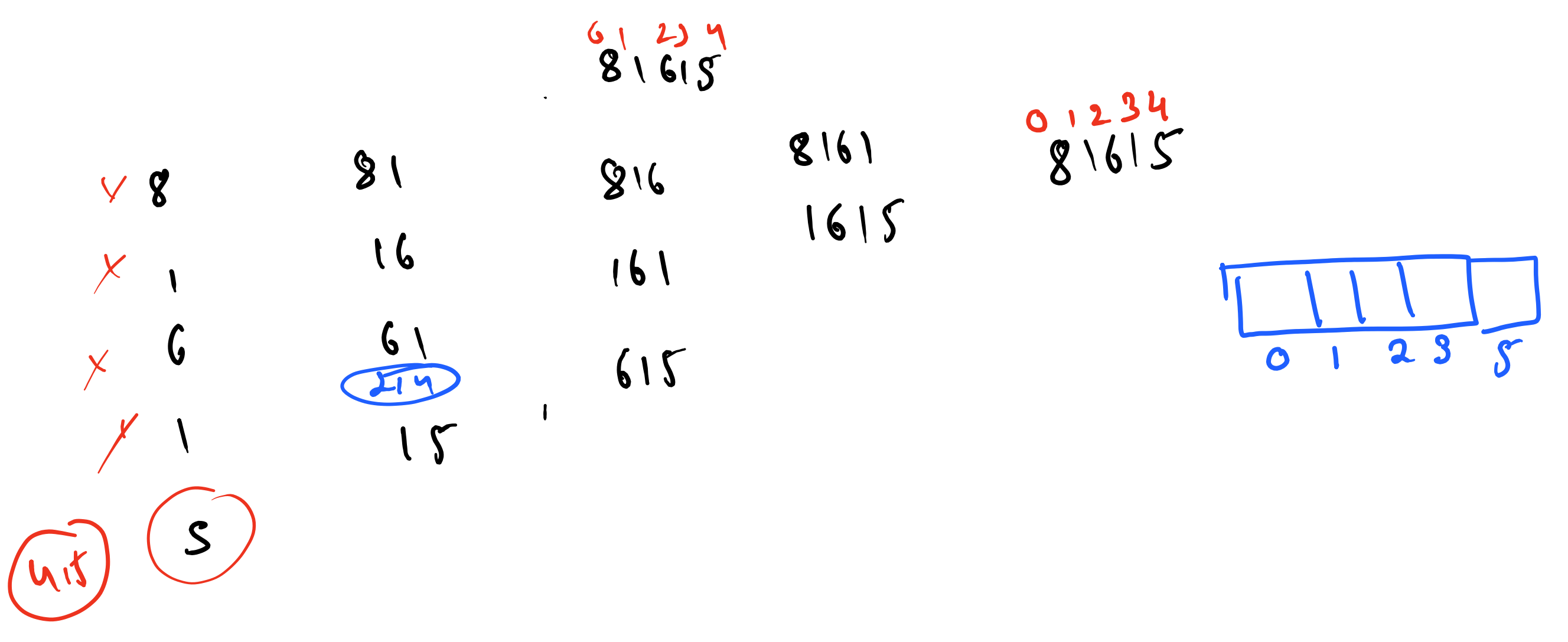
3. Maximization Goal:

Since multiple solutions may exist, the goal is to find the **maximum number** of CB numbers that can be extracted from the given string.

Kanak Bhaiya has a class of Launchpad students to teach and needs help solving this challenge. Assist him in finding the solution.

4991
499
991


81615
8
81
816
8161
81615
1
16
161
1615
6
61
615
1
15
5
127
2
1
12
127
7
27

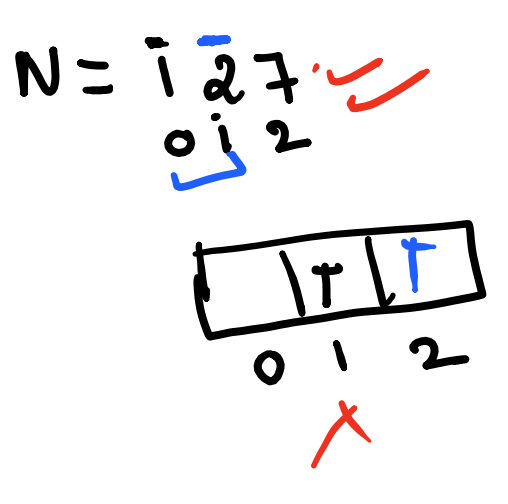

0 1 2 3 4
81615
8
81
816
8161
81615
1
16
161
1615
6
61
615
1
15
0 1 2 3 5
4,1,5
5

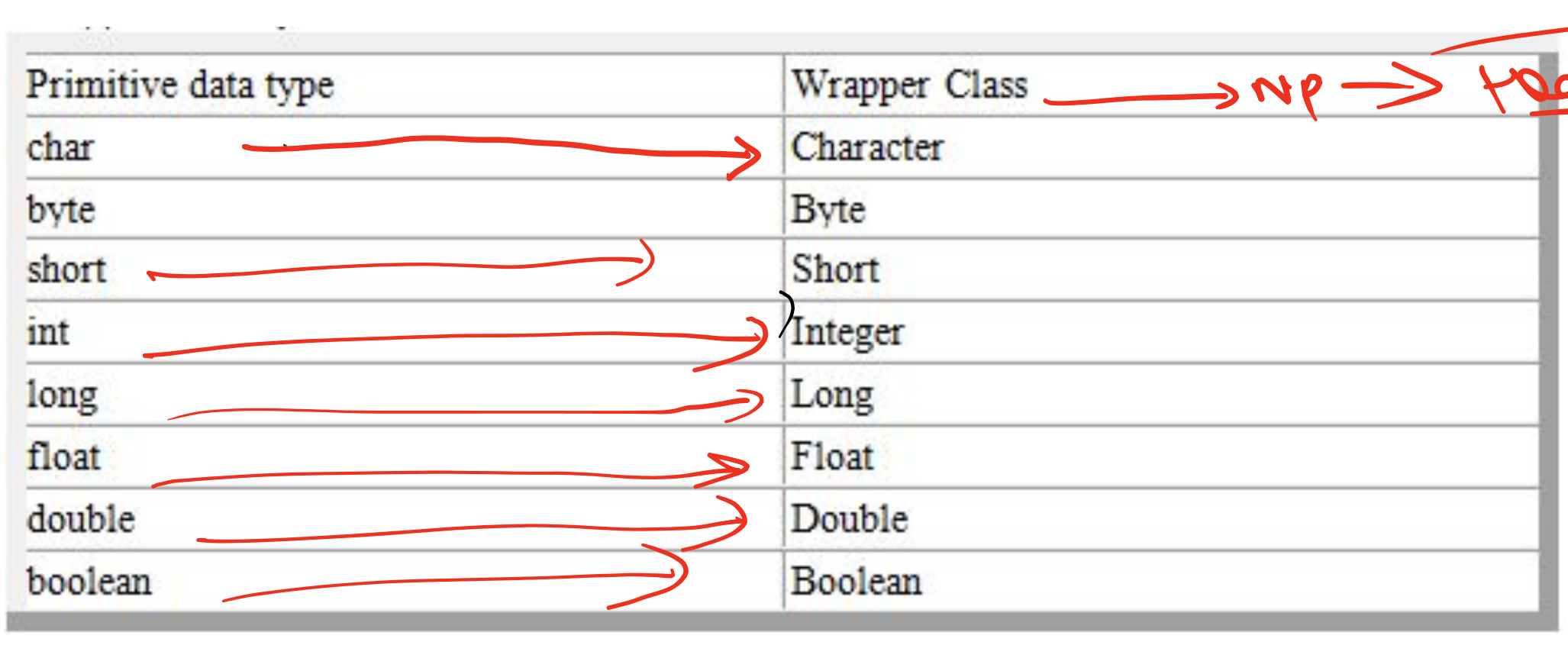
```java
public static void SubString(String s) {
        int c = 0;
        boolean[] visited = new boolean[s.length()];
        for (int len = 1; len <= s.length(); len++) {
            for (int j = len; j <= s.length(); j++) {
                int i = j - len;
                String str = s.substring(i, j);// i to j-1
                long n = Long.parseLong(str);
                if (IsCbnumber(n) && isivisited(visited, i, j - 1)) {
                    c++;
                }
            }
        }
    }

    public static boolean isivisited(boolean[] visited, int si, int ei) {
        // TODO Auto-generated method stub
        for (int i = si; i <=ei; i++) {
            if(visited[i]==true) {
                return false;
            }
        }
        for (int i = si; i <=ei; i++) {
            visited[i]=true;
        }
        return true;
    }
```

c = 1 2

→ len = 1

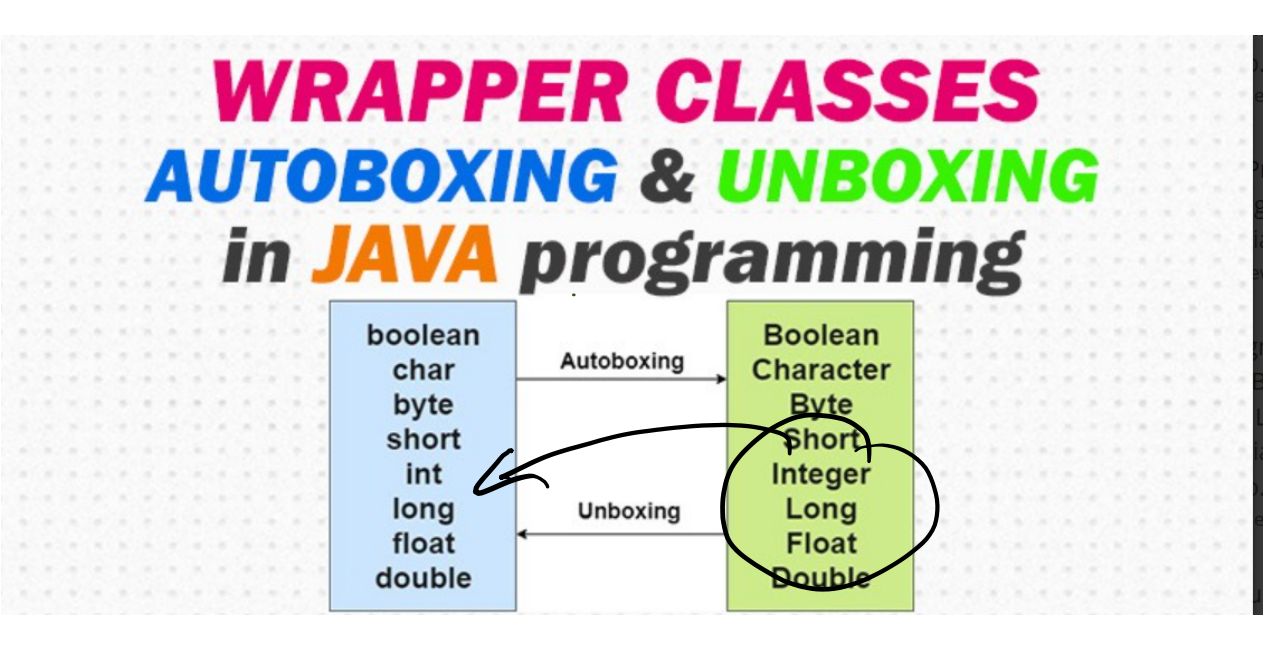
len = 3, j = 3, i = 0


N = 127
0 1 2
T T
0 1 2

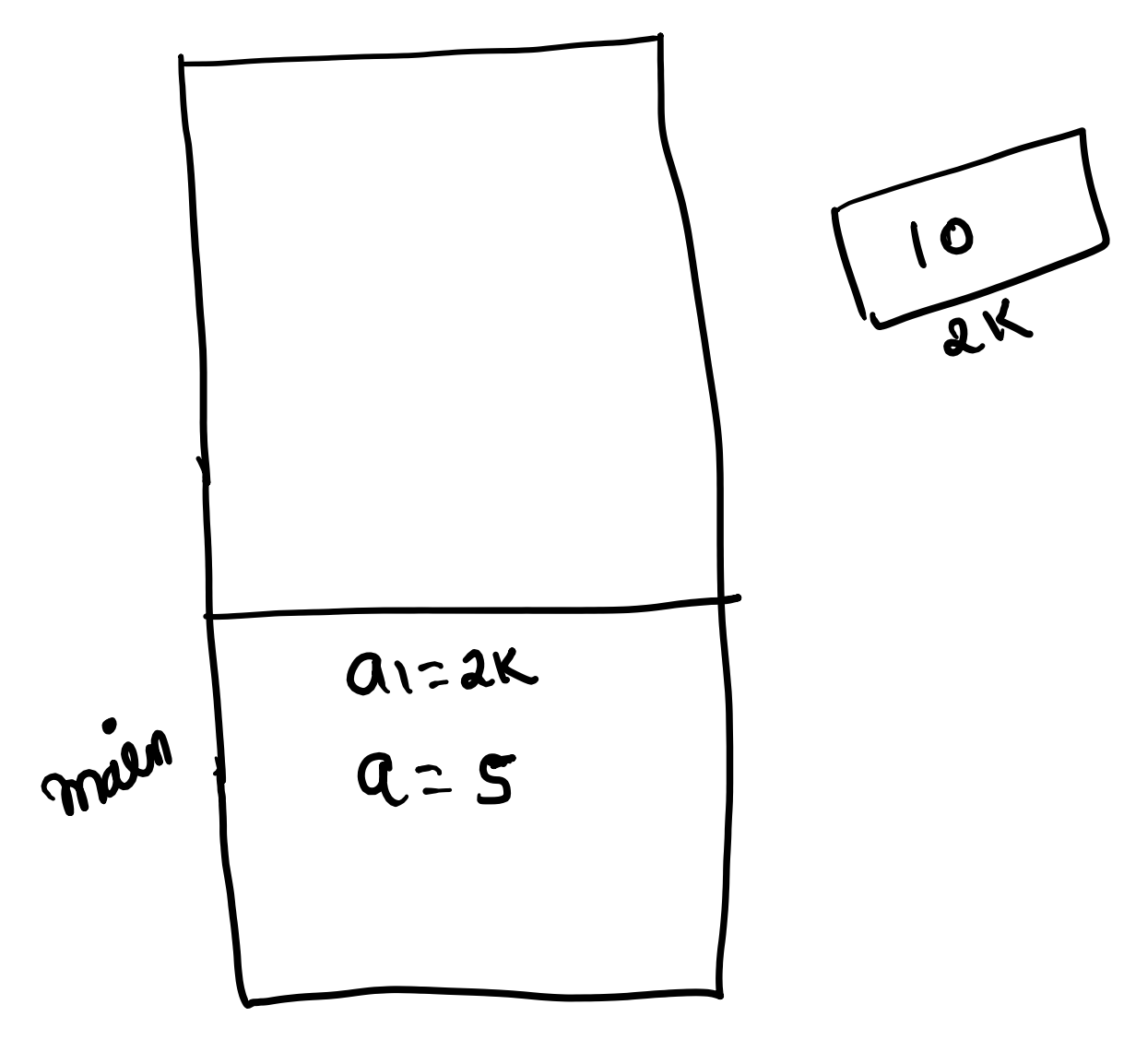


Primitive data type	Wrapper Class
char	Character
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
boolean	Boolean

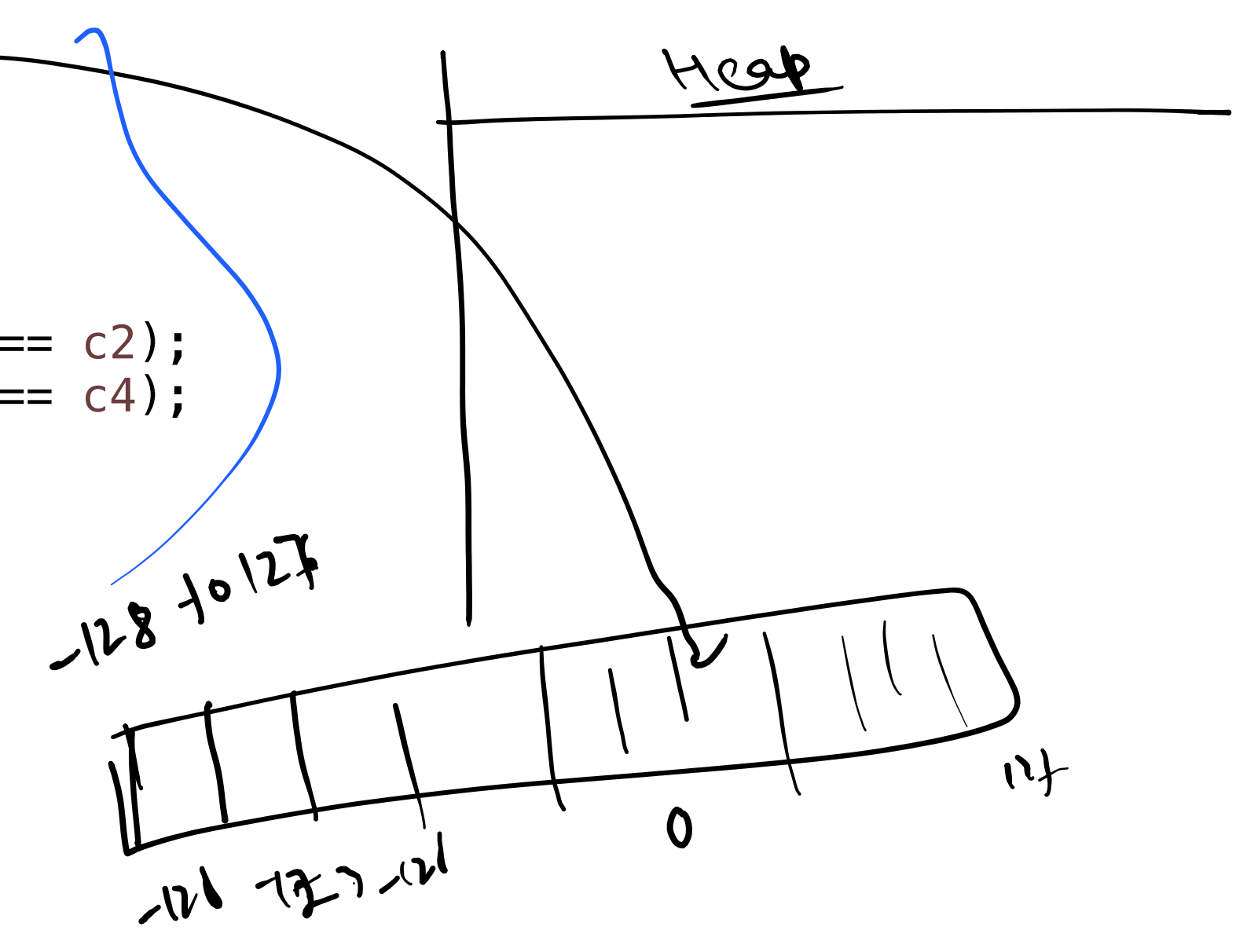
→ NP → Heap


WRAPPER CLASSES
AUTOBOXING & UNBOXING
in JAVA programming
boolean
char
byte
short
int
long
float
double
Autoboxing
Unboxing
Boolean
Character
Byte
Short
Integer
Long
Float
Double

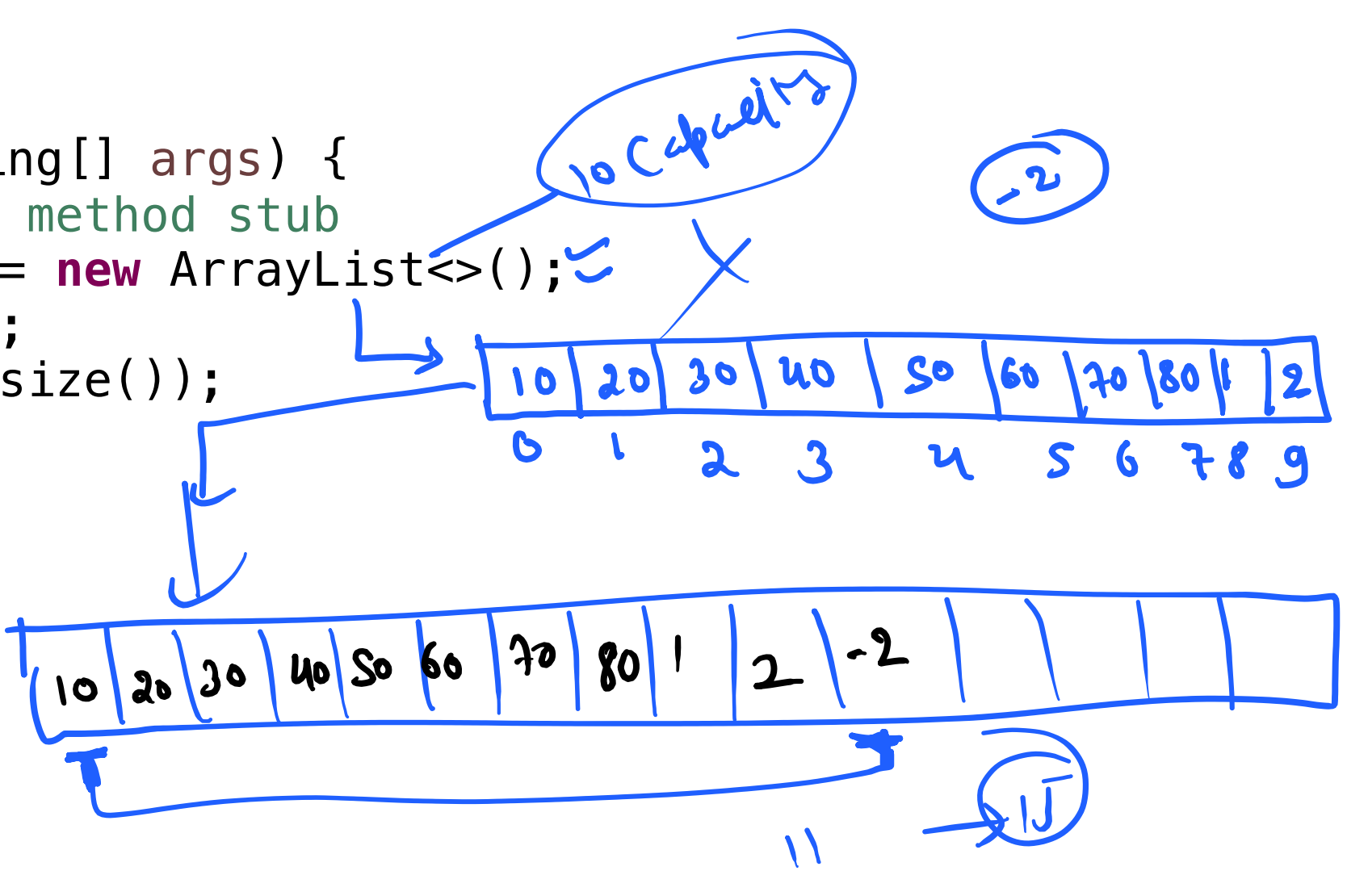
```java
public static void main(String[] args) {
		// TODO Auto-generated method stub
		Integer a1 = 10;
		int a = 5;
		System.out.println(a1);
		System.out.println(a);

	}
```

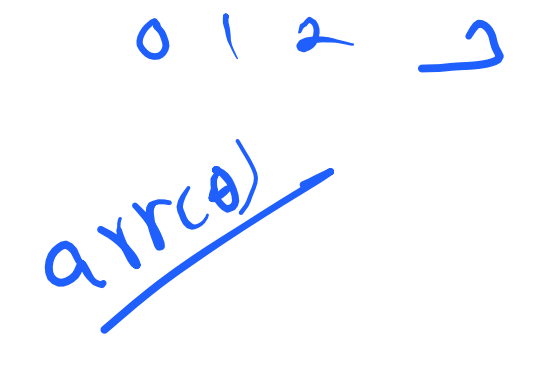

10
2K
main
a1=2K
a = 5

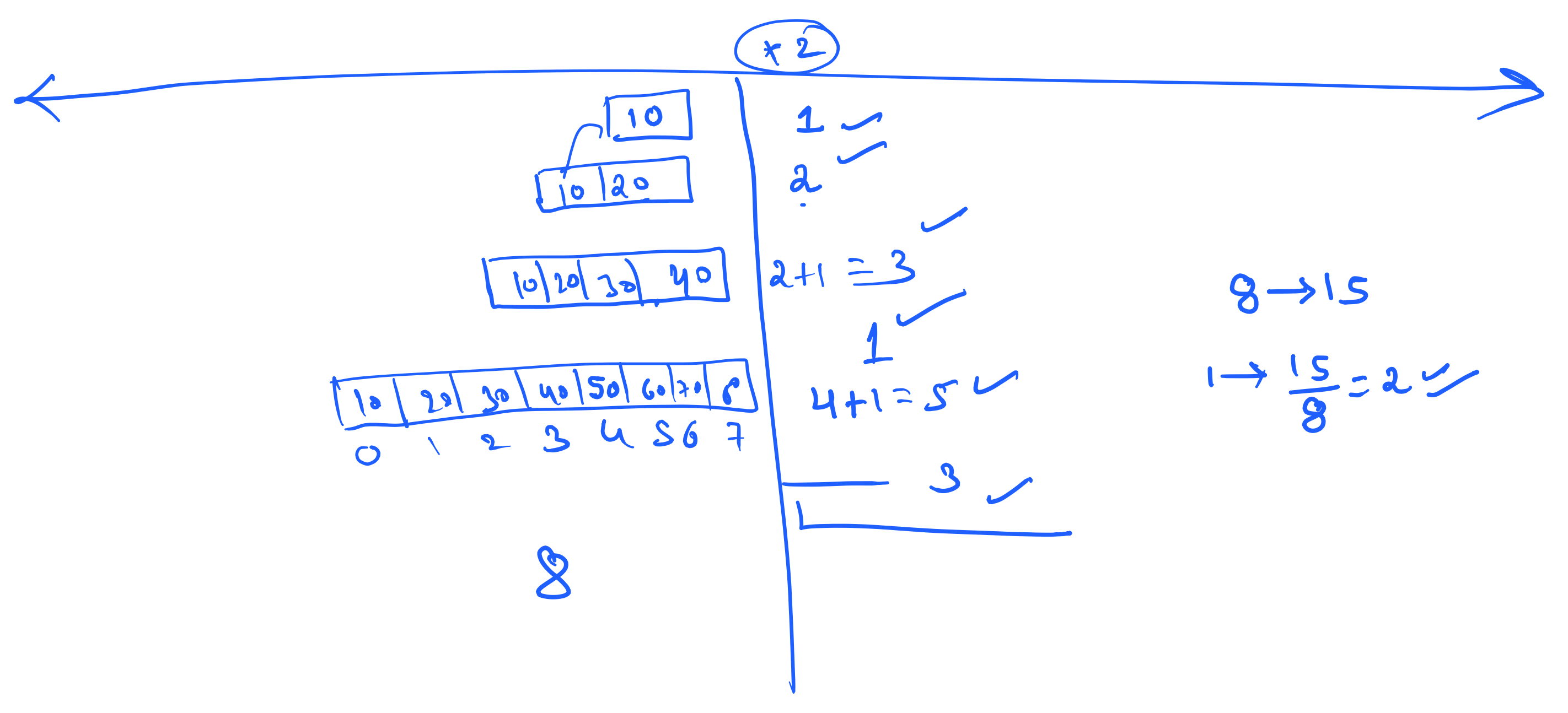
```java
Integer c1 = 19;
Integer c2 = 19;
Integer c3 = 191;
Integer c4 = 191;
System.out.println(c1 == c2);
System.out.println(c3 == c4);
```


Heap
-128 to 127
-128
0
127

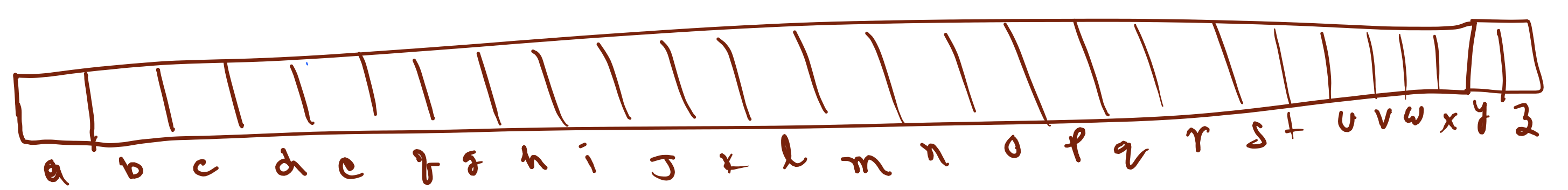
```java
public static void main(String[] args) {
		// TODO Auto-generated method stub
		ArrayList<Integer> ll = new ArrayList<>();
		System.out.println(ll);
		System.out.println(ll.size());
	}
```


10 capacity
-2
10 20 30 40 50 60 70 80 1 2
0 1 2 3 4 5 6 7 8 9
10 20 30 40 50 60 70 80 1 2 -2
11
15


10 20 20 7
0 1 2 3
arr(0)


×2
10
10 20
10 20 30 40
10 20 30 40 50 60 70 80
0 1 2 3 4 5 6 7
1
2
2+1 = 3
4+1 = 5
3
8
8 → 15
1 → 15/8 = 2

```java
String s= "sasbbxmbnvxnvnabmbscjhgfhgsfghjg";
```


a b c d e f g h i j k l m n o p q r s t u v w x y z